

Problema: conjunto de Mandelbrot

- Para cada pixel c : $z = z^2 + c$ até $|z| > 2$ ou 2000 iterações
- Pixels independentes: paralelizamos o laço das linhas, cada thread escreve a sua faixa da imagem
- Custo por pixel varia muito: o interior do conjunto (centro da imagem) roda as 2000 iterações, as bordas escapam rápido
- Diretiva: `#pragma omp parallel for schedule(runtime) reduction(+:checksum)`
- Máquina: atlantica05, 2× Xeon E5520 (8 núcleos, 16 com HT), gcc -O2, mediana de 3 execuções
- Checksum igual em todas as execuções (paralelo = sequencial)

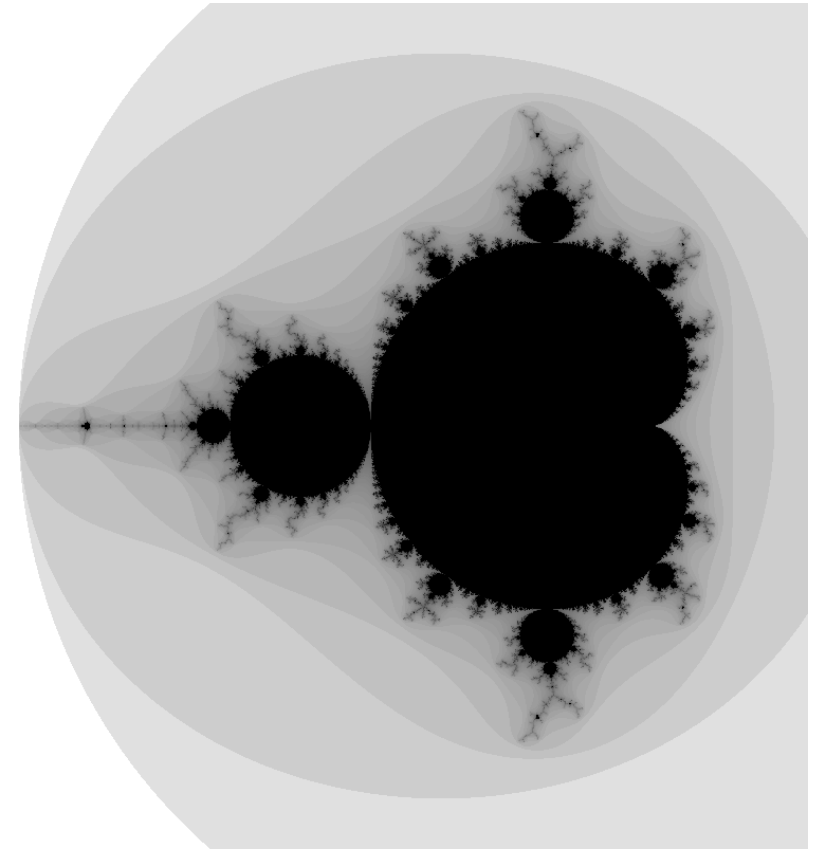
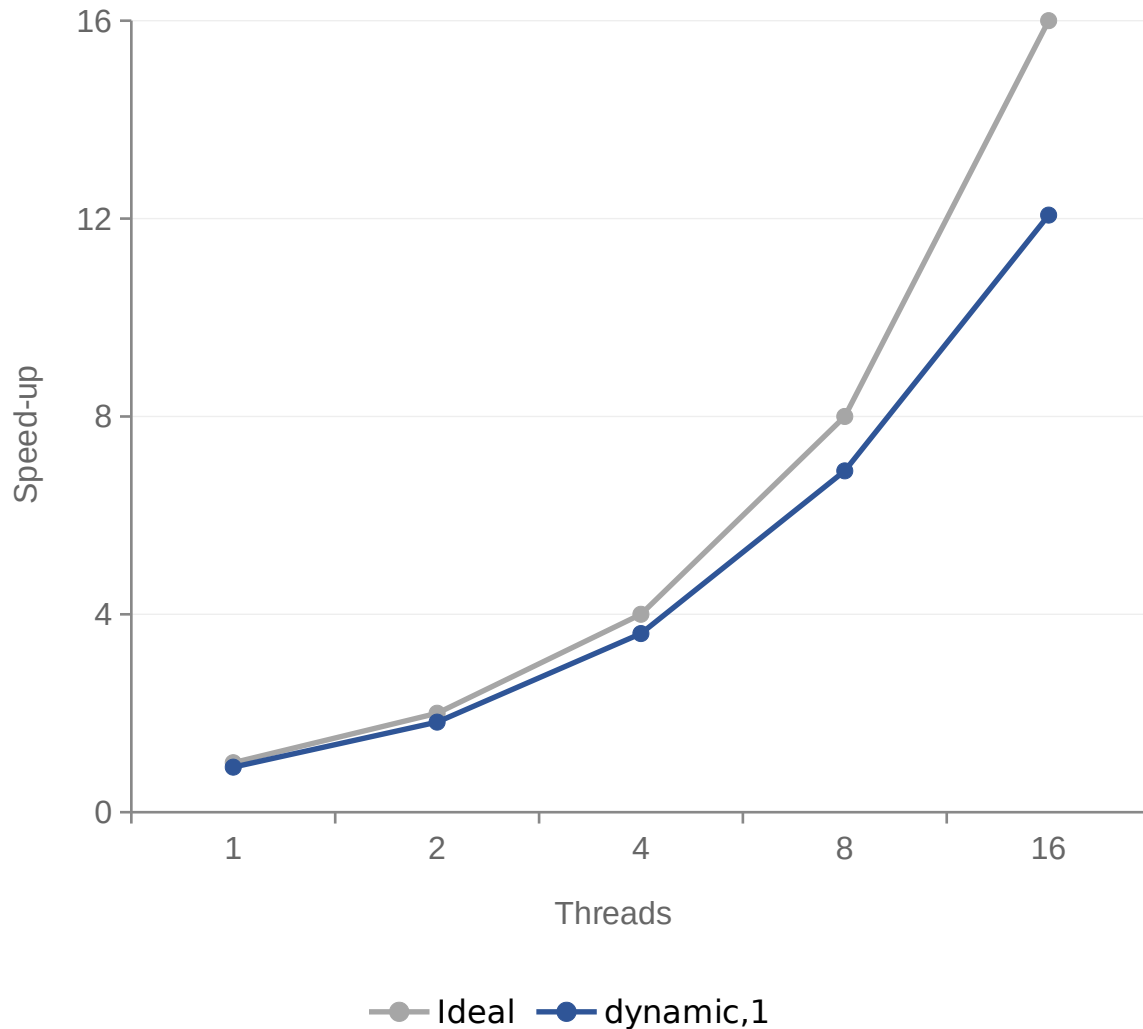


Imagem gerada (tons = nº de iterações)

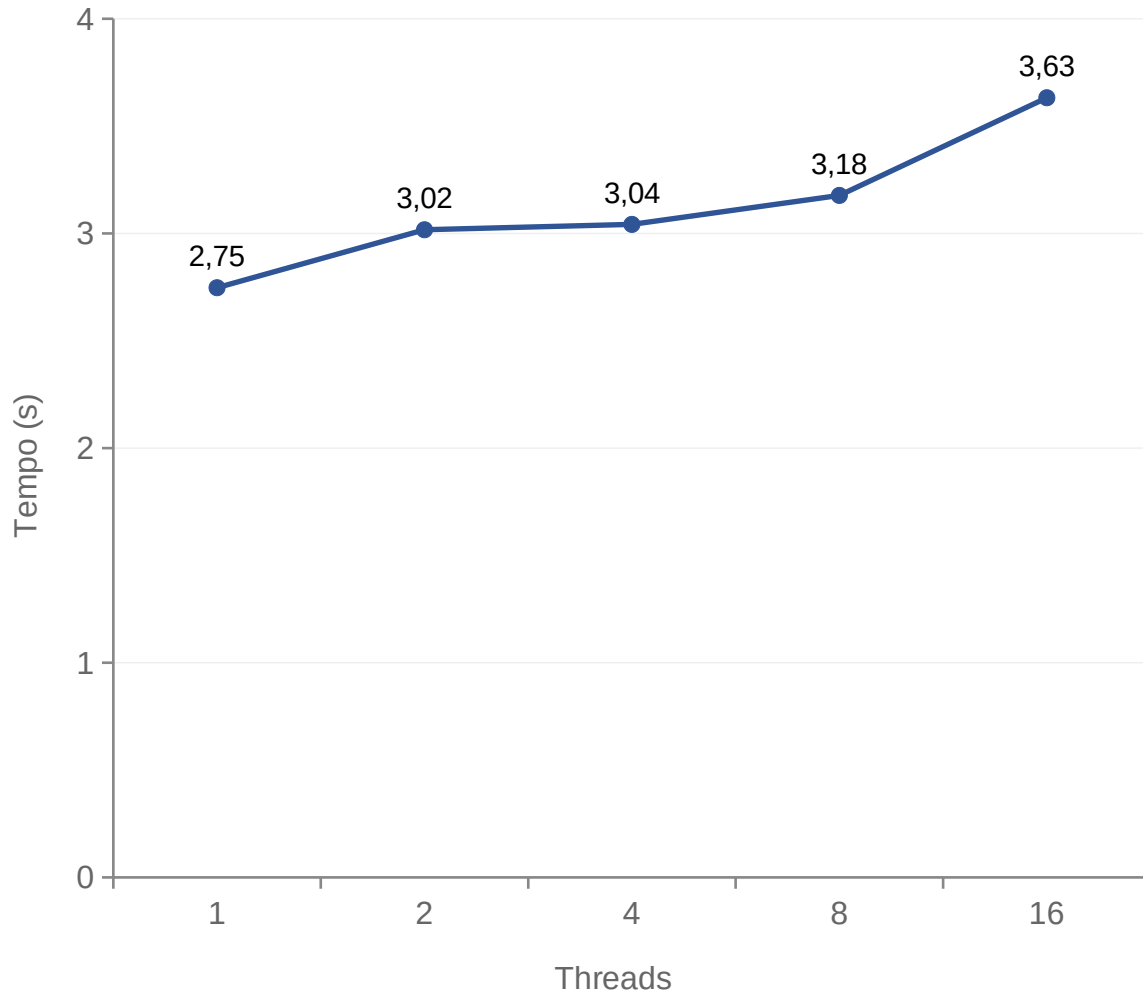
Escalabilidade forte (imagem fixa 4096×4096)



Versão	Threads	Tempo (s)	Speed-up	Efic. (%)
sequencial	1	43,84	-	-
dynamic,1	4	12,15	3,61	90,2
dynamic,1	8	6,35	6,90	86,3
dynamic,1	16 (HT)	3,63	12,07	75,4
guided	8	9,60	4,56	57,1
static	8	14,94	2,93	36,7

- dynamic,1 fica perto do ideal até 8 threads
- static perde porque as linhas caras do centro caem sempre nas mesmas threads (com chunk 1 ele volta a 6,37 s)
- Com HT o ganho continua, mas a eficiência cai

Escalabilidade fraca (1024×1024 pixels por thread)



Threads	Imagem	Tempo (s)	Efic. (%)
1	1024×1024	2,75	100,0
2	1448×1448	3,02	91,0
4	2048×2048	3,04	90,3
8	2896×2896	3,18	86,5
16 (HT)	4096×4096	3,63	75,6

- O ideal seria o tempo constante
- Até 8 threads o tempo sobe pouco (8× mais pixels, +16% de tempo)
- Em 16 threads a queda vem do hyper-threading
- Conclusão: com dynamic,1 o programa escala bem até os 8 núcleos físicos